

Documentación General del Proyecto

Plant-It

Visión General

- **Stack:**
 - Frontend: React (JSX), React Router, Zustand, Axios.
 - Backend: Node.js + Express, MongoDB + Mongoose, Zod, JWT (cookies httpOnly).
 - BLE (Web Bluetooth) para configurar ESP32 y enviarle config inicial.
- **Qué hace:**
 - Administra plantas por usuario.
 - Configura la ESP32 vía BLE (WiFi + API base + IDs).
 - Recibe telemetría de la ESP32 por HTTP y actualiza las lecturas.
 - Muestra “estado de ánimo” de la planta según rangos por tipo.

Arquitectura y Flujo End-to-End

- **Autenticación:**
 - Registro/Login en backend → genera JWT guardado en cookie httpOnly.
 - Frontend usa `useAuthStore` para guardar usuario en memoria y `axios` con `withCredentials`.
- **Datos y Estado:**
 - Backend expone API REST para CRUD de plantas y obtención de perfil de usuario.
 - Frontend usa `usePlantStore` para lista, planta activa y metadatos locales (color/tipo).
- **BLE y Configuración de la ESP32:**
 - Se empareja con ESP32 (Web Bluetooth).

- Se envía WiFi y luego se envía configuración con `plantId`, `deviceId` y `apiBase`.
- **Telemetría:**
 - La ESP32 hace POST a `/api/plants/ingest/:plantId` con `deviceId`.
 - Backend valida `deviceId` y actualiza lecturas (humedad, luz, temperatura, batería).
 - Frontend hace polling para refrescar datos de la planta activa.

Frontend

Estructura

- `src/main.jsx`
Monta App con React Router y estilos globales.
- `src/App.jsx`
Rutas: Welcome, Login, Register, PlantList, PlantHome, Settings.
Protege rutas con verificación de sesión.
Carga inicial de plantas y muestra `BottomNav` cuando corresponde.

Estado Global (Zustand)

- `store/useAuthStore.js`
 - `user`: usuario logueado o `null`.
 - `ready`: indica si ya se intentó cargar sesión.
 - Acciones: `setUser`, `setReady`.
- `store/usePlantStore.js`
 - `plants`: lista del usuario.
 - `active`: planta seleccionada en la UI.
 - `meta`: metadatos locales por planta (color maceta, tipo) persistidos en `localStorage`.

- Acciones: `setPlants`, `setActive`, `addPlant`, `setMeta(id, data)`, `loadMeta()`.

API (Axios)

- `api/axios.js`
Cliente Axios con `baseUrl` y `withCredentials: true`.
- `api/auth.js`
register, login, profile, logout.
- `api/plants.js`
createPlant, getPlants, getPlantById, deletePlant, updatePlantName.

Páginas

- `pages/Welcome.jsx`
Pantalla de entrada con CTA.
- `pages/Login.jsx`
Envía email/contraseña, guarda usuario, redirige a `/home`.
- `pages/Register.jsx`
Crea usuario y redirige a `/home`.
- `pages/PlantList.jsx`
Carga plantas, siembra `meta`, navega a `/home`, permite crear plantas.
- `pages/PlantHome.jsx`
Polling cada 5s.
Cálculo de ánimo con `getPlantMood`.
Botón para reenviar config a la ESP32.
Modales de info y creación.

Componentes

- `BottomNav.jsx`: navegación inferior.
- `CreatePlantModal.jsx`: flujo BLE completo (conectar, crear, enviar WiFi, enviar config, actualizar stores).
- `PlantInfoModal.jsx`: métricas, recomendaciones, borrar planta.
- `PlantAvatar.jsx`: SVG según mood.

- `SensorCard.jsx`: tarjeta simple.

Utils y Constants

- `utils/plantRanges.js`
Rangos ideales y función `getPlantMood`.
- `constants/plants.js`
Tipos de planta para selects.
- `constants/ble.js`
UUIDs y nombre del dispositivo BLE.

Backend

Core

- `src/app.js`
Express, CORS, JSON, cookies, rutas, error handler, DB, endpoint `/esp-data` opcional.
- `src/server.js`
Arranque del servidor.

Config

- `config/db.js`
Conecta a MongoDB vía Mongoose.

Modelos

- `models/user.js`
`userName, email, password, plants`.
- `models/plant.js`
`name, type, groundHumidity, airHumidity, lightExposure, temperature, batteryLevel, userId, deviceId`.

Controladores

- User controller: `register, login, logout, getProfile, updateUserName, deleteUser`.
- Plant controller: `createPlant, ingestTelemetry, getPlants, getPlantById, updateNamePlant, deletePlant`.

Middlewares

- `validateToken.js`: verifica JWT.
- `validator.middleware.js`: valida body con Zod.
- `errorHandler.js`: errores uniformes.

Rutas

- `user.routes.js`: register, login, logout, profile, update username.
- `plant.routes.js`: ingest, create, get, getById, updateName, delete.

Schemas (Zod)

- `auth.schema.js`: register, login, update username.
- `plant.schema.js`: esquema básico de planta.

Utils

- `jwt.js`: genera access token.
- `appError.js`: clase de error.
- `tokenGenerator.js`: genera secretos para env.

Flujos Clave

Inicio de sesión / registro

Formulario → API → cookie JWT → `useAuthStore` → rutas protegidas.

Creación de planta

Modal → BLE connect → POST `/create` → enviar WiFi → enviar config → stores actualizados.

Polling de datos

Cada 5s → `getPlant` → UI actualizada → `getPlantMood`.

Ingesta ESP32

POST `/ingest/:plantId` con `deviceId` → backend actualiza métricas.

Configuración y Entorno

Variables importantes:

- `MONGODB_URI`
- `ACCESS_TOKEN_SECRET`

- `NODE_ENV` (afecta SameSite y Secure)

Cómo correr localmente

Backend

- Instalar dependencias
- Crear `.env`
- Ejecutar `node server/src/server.js`

Frontend

- Instalar dependencias
- `npm run dev`
- Usar HTTPS o localhost para BLE